

PHYS-467 Machine Learning for Physicists

Exercise Session 13: PyTorch Internals and CNNs

Exercise 1: PyTorch Internals

In this exercise, we will familiarize with PyTorch tensor data structure and automatic differentiation engine. In particular, we will cover some parts of

1. https://pytorch.org/tutorials/beginner/blitz/tensor_tutorial.html;
2. https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html.
3. Given the two tensors $u = [1., 2., 3.]$, $v = [4., 5.]$, and the function $f(u, v) = \sum_i u_i^2 + \sum_i \log v_i$, compute the gradients of f with respect to u and v and compare with your analytical predictions. Define the function f for a second time and compute the two gradients again. What do you find?

Exercise 2: Convolutional Neural Networks

In this exercise, you will train a simple CNN to classify images from the CIFAR10 dataset.

1. Download the CIFAR10 dataset, and build the train and test dataloaders, setting the batch size to 32 and activating reshuffling at each epoch for the train data by setting `shuffle=True`. Visualize some images.
2. Define a `nn.Module` class for a convolutional neural network with a first layer of 6 convolutional channels with filter size 5, a max-pooling layer over a 2×2 window, a second convolutional layer made of 16 channels with filter size 5, another 2×2 max-pooling layer, two dense layers with 120 and 84 neurons respectively, and a final linear layer with 10 outputs.
3. Using the cross-entropy loss and SGD with learning rate 0.01, train the model for 5 epochs. After training, compute the accuracy on the test set.
4. Making an analogy with a physical system, we can think of the negative gradient as a force moving a particle through parameter space, following Newton's laws. Adding a momentum or inertia term, the optimization algorithm remembers the directions of the past gradients and continues to move in their direction. Mathematically,

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t - v_t, \quad (2)$$

where $\gamma \in [0, 1]$ is the momentum parameter, η the learning rate, and θ the parameters of the model. Momentum helps the optimization dynamics gain speed in directions with persistent small gradients and suppresses oscillations. Repeat training, adding `momentum=0.9` to the SGD dynamics.

Exercise 3: Feature Maps

1. Using `torch.fx`, we can visualize the transformations of an input inside our neural network. For different input images, check the outputs of the first convolutional layer, of the first ReLU application, and of the first pooling layer.
2. Can we look at what a particular neuron reacts to? What are the features learned by deep models? A simple idea to visualize these features, called activation maximization, consists in looking for the input with bounded norm that maximizes the activation of a given neuron.¹ Open <https://distill.pub/2017/feature-visualization/appendix/> and check how the neurons in different layers of the GoogLeNet network are specializing to recognize features with various complexity, from simple textures to meaningful semantic concepts!

¹ $x^* = \arg \max_{x: \|x\|=1} h_i^{\ell}(x, \theta^*)$, where h_i^{ℓ} is the activation of the neuron i at layer ℓ of a trained network.